Polynomial Regression on Salary Dataset

1. Collect Data

In [12]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

dataset = pd.read_csv("Salary_Data.csv")
```

2. Data Preprocessing

2.1 Understand the Dataset

In [15]:
```
dataset.head()
```

Out[15]:

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

In [17]:
```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     float64
dtypes: float64(2)
memory usage: 612.0 bytes
```

In [19]:
```
dataset.describe()
```

Out[19]:

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

2.2 Check Missing Values

In [22]:
```
dataset.isnull().sum()
```

Out[22]:
```
YearsExperience    0
Salary             0
dtype: int64
```

No missing values found. No preprocessing required.

2.3 Handle Missing Values

No preprocessing required.

2.4 Encoding

Encoding is not required because all features are numerical.

2.5 Split Independent and Dependent Variables

In [28]:
```
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

In [30]:
```
X
```

Out[30]:
```
array([[ 1.1],
       [ 1.3],
       [ 1.5],
       [ 2. ],
       [ 2.2],
       [ 2.9],
       [ 3. ],
       [ 3.2],
       [ 3.2],
       [ 3.7],
       [ 3.9],
       [ 4. ],
       [ 4. ],
       [ 4.1],
       [ 4.5],
       [ 4.9],
       [ 5.1],
       [ 5.3],
       [ 5.9],
       [ 6. ],
       [ 6.8],
       [ 7.1],
       [ 7.9],
       [ 8.2],
       [ 8.7],
       [ 9. ],
       [ 9.5],
       [ 9.6],
       [10.3],
       [10.5]])
```

2.6 Split Training and Testing Data

In [33]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

2.7 Feature Scaling

Feature scaling is **not required** for Polynomial Regression with Linear Regression.

3. Model Building

3.1 Import Required Libraries

In [36]:
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
```

3.2 Fit the Model

In [38]:
```
poly = PolynomialFeatures(degree=10)

X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

model = LinearRegression()
model.fit(X_train_poly, y_train)
```

Out[38]:
```
LinearRegression()
```

In [39]:
```
model.coef_
```

Out[39]:
```
array([ 0.00000000e+00,  3.06145092e+06, -3.76093852e+06,  2.50121009e+06,
       -1.00338644e+06,  2.56208478e+05, -4.25800962e+04,  4.58812288e+03,
       -3.09123558e+02,  1.18360076e+01, -1.96639585e-01])
```

4. Test the Model

In [42]:
```
y_pred = model.predict(X_test_poly)

salary = model.predict(poly.transform([[6.5]]))
print("Predicted Salary:", salary[0])
```

```
Predicted Salary: 93641.31756019592
```

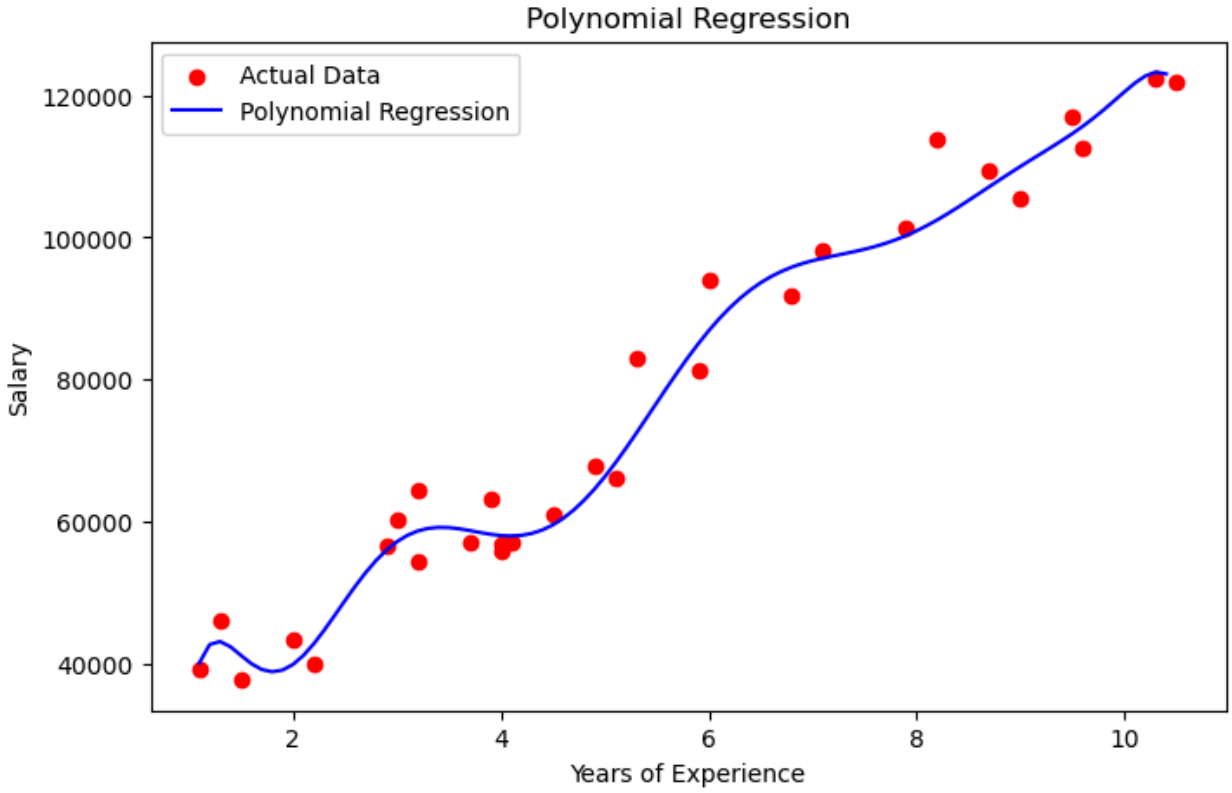
Visualize Polynomial Regression

X_grid = np.arange(min(X), max(X), 0.1).reshape(-1,1) creates many closely spaced input values between the minimum and maximum experience, allowing the polynomial model to make predictions at many points so that the plotted regression curve appears smooth instead of jagged.

In [ ]:
```
```

In [49]:
```
X_grid = np.arange(X.min(), X.max(), 0.1).reshape(-1,1)

plt.figure(figsize=(8,5))
plt.scatter(X, y, color='red', label='Actual Data')
plt.plot(X_grid,
         model.predict(poly.transform(X_grid)),
         color='blue',
         label='Polynomial Regression')
plt.title("Polynomial Regression")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.legend()
plt.show()
```



5. Evaluation Metrics

In [52]:
```
from sklearn.metrics import r2_score
print("\nDegree 10 (Complex)")
print("Train R2:", r2_score(y_train, model.predict(X_train_poly)))
print("Test  R2:", r2_score(y_test, model.predict(X_test_poly)))
```

```
Degree 10 (Complex)
Train R2: 0.9877814369986833
Test  R2: 0.9026933486383973
```

Why does overfitting occur?

As the polynomial degree increases, the model becomes increasingly flexible and begins fitting noise in the training data instead of the true underlying pattern. This results in excellent training performance but poorer generalization on unseen data.

Ridge (L2 Regularization)

- Adds the squared magnitude of coefficients to the loss function.
- Formula:

Loss = MSE + α Σ(w²)

- Shrinks coefficients toward zero but rarely makes them exactly zero.

Lasso (L1 Regularization)

- Adds the absolute value of coefficients to the loss function.
- Formula:

Loss = MSE + α Σ|w|

- Can shrink some coefficients exactly to zero, effectively performing feature selection.

Key Difference

Method	Penalty	Effect
Polynomial Regression	None	May overfit for high degrees
Ridge (L2)	Squares coefficients	Shrinks all coefficients
Lasso (L1)	Absolute coefficients	Removes less useful coefficients

6. Apply Ridge Regression

Why Feature Scaling Now?

Polynomial Regression alone does not require scaling.

However, **Ridge** and **Lasso** penalize the magnitude of coefficients. If one feature has a much larger scale than another, the penalty becomes unfair. Therefore, scaling is recommended before applying Ridge and Lasso.

In [63]:
```
# ====================================
# 6. RIDGE (L2)
# ====================================
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import Ridge,Lasso
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score
#In scikit-learn, a Pipeline allows you to combine multiple preprocessing and
#First create polynomial features → then scale them → then apply Ridge Regress
ridge_model = Pipeline([
    ('poly', PolynomialFeatures(degree=10)),
    ('scaler', StandardScaler()),
    ('ridge', Ridge(alpha=1))
```

```
])

# Train
ridge_model.fit(X_train, y_train)

# Predictions
ridge_train_pred = ridge_model.predict(X_train)
ridge_test_pred = ridge_model.predict(X_test)

# Evaluation
print("\nRidge (degree=10, alpha=1)")
print("Train R2:", r2_score(y_train, ridge_train_pred))
print("Test R2 :", r2_score(y_test, ridge_test_pred))
```

```
Ridge (degree=10, alpha=1)
Train R2: 0.9667535214527384
Test R2 : 0.8934384785752252
```

7. Apply Lasso Regression

In [66]:
```
lasso_model = Pipeline([
    ('poly',PolynomialFeatures(degree=10)),
    ('scaler',StandardScaler()),
    ('lasso',Lasso(alpha=1000,max_iter=10000))
])

# Train
lasso_model.fit(X_train, y_train)

# Predictions
lasso_train_pred = lasso_model.predict(X_train)
lasso_test_pred = lasso_model.predict(X_test)

# Evaluation
print("\nLasso (degree=10, alpha=1000)")
print("Train R2:", r2_score(y_train, lasso_train_pred))
print("Test R2 :", r2_score(y_test, lasso_test_pred))
```

```
Lasso (degree=10, alpha=1000)
Train R2: 0.9636844451149937
Test R2 : 0.8955040548377098
```

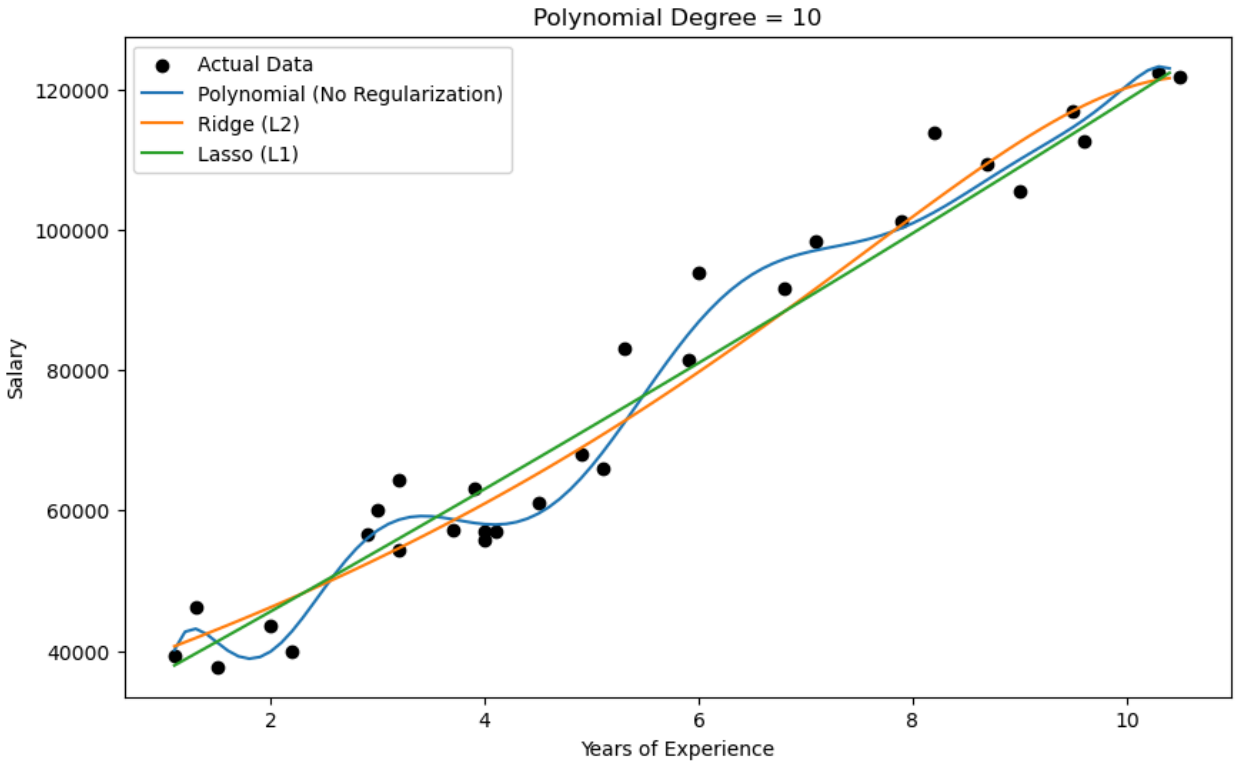
Model	Train R^2	Test R^2	Gap	Result
Degree 10 Polynomial	0.9878	**0.9027**	0.0851	High performance, but **some overfitting**
Ridge (α=1)	0.9668	0.8934	0.0733	**Reduces overfitting**
Lasso (α=1000)	0.9637	0.8955	**0.0682**	**Strongest complexity control**

8. Visual Comparison

In [70]:
```
plt.figure(figsize=(10,6))
plt.scatter(X,y,color='black',label='Actual Data')

plt.plot(X_grid,model.predict(poly.transform(X_grid)),label='Polynomial (No Re
plt.plot(X_grid,ridge_model.predict(X_grid),label='Ridge (L2)')
plt.plot(X_grid,lasso_model.predict(X_grid),label='Lasso (L1)')

plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.title(f'Polynomial Degree = 10')
plt.legend()
plt.show()
```



In [ ]: